

ValiAutoFlow — Documentación Técnica: Instrucciones de Despliegue

Versión: 1.0 · Última actualización: 2026-08-03

Complementa a [ARQUITECTURA.md](#) y [OPERACION.md](#).

Este documento explica cómo publicar cambios a producción de forma segura, y cómo levantar el sistema desde cero.

1. Entorno de producción (dónde vive todo)

Elemento	Valor
Carpeta de la app	C:\Hosting\s704ag
Servicio (proceso)	NSSM amadsite_s704ag (usuario restringido)
Puerto interno	3105 (next start -p 3105)
Binario NSSM	C:\tools\nssm.exe
Proxy	Apache 2.4 → 127.0.0.1:3105
TLS/DNS	Cloudflare
Dominio	https://valiautoflow.com
Base de datos	MySQL db_s704ag (credenciales en DATABASE_URL del .env)
Log de la app	C:\Hosting\s704ag\amad-service.log

IMPORTANTE: la app corre bajo un servicio NSSM con usuario restringido. **PM2 ya NO la maneja**. Nunca recrear la app en PM2.

2. Reglas de oro (leer antes de tocar)

- NUNCA** ejecutar prisma generate , npm install ni prisma migrate dentro de C:\Hosting\s704ag — el usuario del servicio no tiene permisos y da EPERM . Esas operaciones se hacen en el entorno de desarrollo.
- El **deploy es en el sitio** (build en C:\Hosting\s704ag) seguido de reinicio del servicio NSSM. No hay pipeline automático que publique al servidor Windows (el CI de GitHub Actions valida, no despliega).
- Siempre reiniciar el servicio** tras cambiar código o .env : los cambios de .env solo se leen al arrancar; el código servido es el de .next generado por el build.
- Tras desplegar, **verificar**: login HTTP 200 + WhatsApp CONNECTED en el log.
- Detrás de Apache hay **timeout ~30s**: cualquier endpoint que tarde más debe ser **asíncrono** (responder al instante y trabajar en segundo plano), como la generación de video/reels.

3. Despliegue estándar (cambios de código)

Desde C:\Hosting\s704ag :

```
# 1) Detener el servicio y matar procesos node residuales del puerto/servicio
C:\tools\nssm.exe stop amadsite_s704ag
Get-Process node -ErrorAction SilentlyContinue |
  Where-Object { $_.Path -like '*s704ag*' } | Stop-Process -Force -ErrorAction SilentlyContinue

# 2) Compilar (Next build). NO correr prisma generate / npm install aquí.
npm run build

# 3) Arrancar el servicio de nuevo
C:\tools\nssm.exe start amadsite_s704ag

# 4) Esperar ~40s y verificar
Start-Sleep -Seconds 42
Invoke-WebRequest -Uri 'https://valiautoflow.com/login' -UseBasicParsing -TimeoutSec 20 |
  Select-Object -ExpandProperty StatusCode # esperado: 200
```

Verificación adicional de WhatsApp (que reconectó sin problemas):

```
Get-Content C:\Hosting\s704ag\amad-service.log -Tail 40 | Select-String 'CONNECTED|Reply sent'
```

Si el build **falla**, el servicio queda detenido y el sitio devuelve **502/503**. Corregir el error de compilación (TypeScript/ESLint) y repetir. El build imprime el árbol de rutas al final cuando termina bien.

4. Cambios que solo tocan `.env`

`.env` se lee **al arrancar**. Basta reiniciar (sin rebuild) para variables de **servidor**:

```
C:\tools\nssm.exe restart amadsite_s704ag
```

Excepción: las variables ****NEXT_PUBLIC_*** se "hornean" en el bundle durante `npm run build`; si cambias una de esas, hay que rebuild**, no solo reiniciar.

5. Cambios en la base de datos (esquema)

El esquema (`prisma/schema.prisma`) **no** se migra dentro de Hosting (EPERM). Flujo correcto:

1. En **desarrollo**: editar `prisma/schema.prisma`, correr `npm run prisma db push` (o `migrate`) contra la BD, y `prisma generate`.
2. Llevar el código actualizado (incluye el cliente Prisma regenerado y `schema.prisma`) a `C:\Hosting\s704ag`.
3. Hacer el **despliegue estándar** (§3).

Cambios de datos puntuales/seguros pueden hacerse con scripts Node que carguen el `.env` y usen `PrismaClient` directamente (patrón usado en scripts de mantenimiento). Evitar operaciones destructivas sin respaldo.

6. Assets estáticos (imágenes, videos del Manual)

Los archivos en `public/` se sirven directamente en la raíz del dominio (`/...`) **sin rebuild**. Ejemplo: los videos del Manual (`public/manual/*.mp4`) se sirven en `https://valiautoflow.com/manual/<id>.mp4`. Basta copiarlos a `public/manual/`; si el componente ya los referencia, aparecen de inmediato (puede requerir refrescar caché del navegador).

Regenerar los videos guía del Manual: script de grabación (Playwright → ffmpeg) que produce cada `public/manual/<modulo>.mp4` a 1920×1080. Ejecutar con `node <grabador>.mjs` y verificar dimensiones con ffmpeg.

7. Levantar el sistema desde cero (recuperación / servidor nuevo)

Requisitos: Windows Server, Node.js 24, MySQL, Apache 2.4, Cloudflare configurado, NSSM (`C:\tools\nssm.exe`).

1. **Código:** clonar el repo o copiar `C:\Hosting\s704ag` (sin `node_modules` ni `.next`).
2. **Dependencias** (en un entorno con permisos, p. ej. la carpeta de desarrollo): `npm ci` (o `npm install`) y `npx prisma generate` .
3. **.env** : colocar el archivo con todas las variables (ver ARQUITECTURA §11). Como mínimo: `DATABASE_URL` , `NEXTAUTH_SECRET` , `NEXTAUTH_URL` , `NEXT_PUBLIC_APP_URL` , `PORT=3105` , `CRON_SECRET` , llaves de IA (`MINIMAX_*` , `GROQ_API_KEY`) e integraciones.
4. **Base de datos:** crear `db_s704ag` en MySQL; `npx prisma db push` para crear el esquema. Restaurar datos desde respaldo si aplica.
5. **Build:** `npm run build` .
6. **Servicio NSSM:**

```
C:\tools\nssm.exe install amadsite_s704ag "C:\Program Files\nodejs\node.exe"
C:\tools\nssm.exe set amadsite_s704ag AppDirectory "C:\Hosting\s704ag"
C:\tools\nssm.exe set amadsite_s704ag AppParameters "node_modules\next\dist\bin\next start -p 3105"
C:\tools\nssm.exe set amadsite_s704ag AppStdout "C:\Hosting\s704ag\.amad-service.log"
C:\tools\nssm.exe set amadsite_s704ag AppStderr "C:\Hosting\s704ag\.amad-service.log"
C:\tools\nssm.exe start amadsite_s704ag
```

(Ajustar rutas según la instalación de Node/Next.)

1. **Apache:** VirtualHost con `ProxyPass / http://127.0.0.1:3105/` y `ProxyPassReverse` (SSL termina en Apache/Cloudflare).
2. **Cloudflare:** DNS del dominio apuntando al servidor (con DDNS si la IP es dinámica).
3. **Tareas programadas (cron):** registrar las 6 tareas `ValiAutoFlow-Cron-*` que ejecutan `scripts\cron-runner.ps1 -Endpoint <x>` (ver ARQUITECTURA §9 para frecuencias).
4. **Verificar:** login 200, WhatsApp CONNECTED, y una corrida de `automations` OK (`.cron-logs\automations.log`).

8. CI (GitHub Actions)

- Repositorio: GitHub `amadsoftwareresolutions-spec/valia-autoflow` .
- Workflow: `.github/workflows/ci-cd.yml` — en push/PR a `main` / `production-ready` corre **lint + typecheck + tests** (Node 24, `ubuntu-latest`).
- El **deploy a producción es manual** (§3); el CI sirve para validar antes de desplegar.
- Seguridad: el `git remote` tiene un token de acceso incrustado en la URL — se recomienda **rotarlo** y usar un credential manager en lugar de dejarlo en la URL.

9. Checklist de despliegue

- [] Build local/desarrollo compila sin errores TS/ESLint.
- [] Si tocaste esquema de BD: `db push` + `generate` hechos en desarrollo.
- [] `nssm stop` + matar node residual.
- [] `npm run build` termina imprimiendo el árbol de rutas.
- [] `nssm start` + esperar ~40s.
- [] `login` responde **200**.
- [] WhatsApp **CONNECTED** en `.amad-service.log` .
- [] Endpoint tocado responde correctamente (probar en producción).
- [] Si aplica, verificar el cron relacionado en `.cron-logs\` .

